```
        "fuzziness": 2          <-- Setting fuzziness to 2 forgives two-letter
      }                             substitutions/modifications.
    }
  }
}
```

This can be a clumsy way to handle things because we may not know if the user has mistyped one letter or a few letters. This is why Elasticsearch provides a default setting for `fuzziness`: the `AUTO` setting. If the `fuzziness` attribute is not supplied, the default setting of `AUTO` is assumed. The `AUTO` setting determines the edit distance based on the length of the word, as shown in table 9.3. Sticking to the default setting of `AUTO` for the `fuzziness` attribute is preferred unless we know the exact use case at hand.

Table 9.3 Using the `AUTO` `fuzziness` setting

Word length in characters	Fuzziness (edit distance)	Explanation
0 to 2	0	If the word is shorter than two characters, fuzziness is not applied. This means misspelled words cannot be corrected.
3 to 5	1	If the length of the word is 3 to 5 characters, an edit distance of 1 is applied.
More than 5	2	If the length of the word is longer than 5 characters, an edit distance of 2 is applied.

Let's wrap up here. We learned a lot about term-level searching in this chapter. While term-level search helps find answers in structured data, the real power of a search engine lies in its ability to search unstructured data. In Elasticsearch, unstructured data is associated with full-text searching—searching `text` fields and yielding results with relevance scores. We examine full-text queries in detail in the next chapter.

Summary

- Term-level searching is carried out on structured data such as numbers, keywords, Booleans, and dates.
- A term-level search produces a binary result: the search leads to a result or none. There is no *likely match* scenario.
- Term-level queries are not analyzed, meaning that when applied to `text` fields undergoing text analysis during indexing, they can yield incorrect or no results.
- There are many term-level search queries: `term`, `terms`, `prefix`, `range`, `fuzzy`, and others.
- A `term` query searches for a single term against a field, while a `terms` (plural) query search for multiple values against a single field.
- A `range` query helps search data within a set of bounds: for example, searching for crimes that happened in London in the last month.